

Lab 11: EVENT HANDLING & FORM CƠ BẢN TRONG REACT

Mục tiêu bài thực hành

Sau bài thực hành, sinh viên có thể:

- Hiểu cách React xử lý sự kiện.
- Sử dụng các event phổ biến như onClick, onChange, onSubmit, onFocus, onBlur.
- Truyền và xử lý sự kiện trong component.
- Xây dựng form cơ bản trong React.
- Hiểu khái niệm **Controlled Component**.
- Hiểu khái niệm **Uncontrolled Component**.
- Biết sử dụng useState để quản lý dữ liệu form.
- Biết sử dụng useRef để truy cập dữ liệu từ form.
- So sánh và lựa chọn Controlled/Uncontrolled phù hợp.

PHẦN 1. CƠ SỞ LÝ THUYẾT

1. Event Handling trong React là gì?

Trong ứng dụng web, **event** là một sự kiện xảy ra khi người dùng tương tác với giao diện.

Ví dụ:

- Nhấn nút.
- Nhập dữ liệu vào ô textbox.
- Chọn một checkbox.
- Chọn một option.
- Submit form.
- Di chuyển chuột.
- Focus vào một input.

React cung cấp cơ chế để chúng ta đăng ký hàm xử lý các sự kiện này.

Ví dụ:

```
<button onClick={handleClick}>  
  Click me
```

```
</button>
```

Khi người dùng nhấn nút, hàm handleClick sẽ được gọi.

2. Cú pháp Event trong React

React sử dụng tên event theo dạng **camelCase**.

HTML thường viết:

```
<button onclick="handleClick()">
```

Trong React:

```
<button onClick={handleClick}>
```

Một số event thường dùng:

Event	Ý nghĩa
onClick	Khi click
onChange	Khi giá trị input thay đổi
onSubmit	Khi submit form
onFocus	Khi input được focus
onBlur	Khi input mất focus
onmouseenter	Khi chuột đi vào
onmouseleave	Khi chuột rời khỏi
onKeyDown	Khi nhấn phím
onKeyUp	Khi nhả phím

PHẦN 2. THỰC HÀNH EVENT HANDLING

Bước 1: Tạo project React

Nếu sử dụng Vite:

```
npm create vite@latest react-event-form -- --template react
```

Di chuyển vào thư mục:

cd react-event-form

Cài đặt package:

```
npm install
```

Chạy project:

```
npm run dev
```

Mở địa chỉ được Vite cung cấp, thường có dạng: <http://localhost:5173>

Bước 2. Xử lý sự kiện onClick

Mở src/App.jsx

Thay nội dung bằng:

```
function App() {  
  const handleClick = () => {  
    alert("Bạn vừa nhấn nút!");  
  };  
  
  return (  
    <div>  
      <h1>Event Handling</h1>  
  
      <button onClick={handleClick}>  
        Click me  
      </button>  
    </div>  
  );  
}  
  
export default App;
```

Giải thích

Đoạn:

onClick={handleClick}

có nghĩa là:

Khi button được click → React gọi hàm handleClick.

Lưu ý quan trọng

Không viết:

onClick={handleClick()}

vì cách này sẽ **gọi hàm ngay khi component render**, thay vì chờ người dùng click.

Thông thường sử dụng:

onClick={handleClick}

hoặc:

onClick={() => handleClick()}

Bước 3. Event Object

React truyền một đối tượng event vào hàm xử lý.

Ví dụ:

```
const handleClick = (event) => {  
  console.log(event);  
};
```

Có thể lấy thông tin từ event:

```
const handleClick = (event) => {  
  console.log(event.target);  
};
```

Ví dụ hoàn chỉnh:

```
function App() {  
  const handleClick = (event) => {  
    console.log("Element:", event.target);  
    console.log("Event:", event);  
  };  
  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}  
  
export default App;
```

event.target chính là element đã phát sinh sự kiện.

Bước 4. Truyền tham số cho Event Handler

Giả sử muốn truyền tên sinh viên:

```
function App() {
  const handleStudentClick = (name) => {
    alert(`Xin chào ${name}`);
  };

  return (
    <div>
      <button onClick={() => handleStudentClick("Nguyễn Văn A")}>
        Sinh viên A
      </button>

      <button onClick={() => handleStudentClick("Trần Văn B")}>
        Sinh viên B
      </button>
    </div>
  );
}

export default App;
```

Ở đây:

`onClick={() => handleStudentClick("Nguyễn Văn A")}`

là một **arrow function** được thực thi khi click.

PHẦN 3. LÀM VIỆC VỚI INPUT

Bước 5. Event onChange

`onChange` thường được sử dụng để theo dõi dữ liệu người dùng nhập.

Ví dụ:

```
function App() {
  const handleChange = (event) => {
    console.log(event.target.value);
  };

  return (
    <div>
      <input
        type="text"
      />
    </div>
  );
}
```

```
        onChange={handleChange}
      />
    </div>
  );
}
```

```
export default App;
```

Nếu nhập:

React

Console sẽ nhận được các giá trị thay đổi:

```
R
Re
Rea
Reac
React
```

Bước 6. Sử dụng useState với Input

Đây là bước rất quan trọng để hiểu **Controlled Component**.

Import useState:

```
import { useState } from "react";
Sau đó:
function App() {
  const [name, setName] = useState("");

  const handleChange = (event) => {
    setName(event.target.value);
  };

  return (
    <div>
      <h1>Controlled Input</h1>

      <input
        type="text"
        value={name}
        onChange={handleChange}
      />

      <p>Tên: {name}</p>
    </div>
```

```
);  
}  
  
export default App;
```

Bước 7. Hiểu Controlled Component

Một input được gọi là **Controlled Component** khi giá trị của input được React quản lý thông qua state.

Mô hình:

```
User nhập  
  ↓  
onChange  
  ↓  
setState  
  ↓  
React State  
  ↓  
Render lại  
  ↓  
value của input
```

Ví dụ:

```
const [name, setName] = useState("");  
  
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
>
```

Ở đây:

```
value={name}
```

là điểm quan trọng.

React giữ quyền kiểm soát giá trị của input.

Bước 8. Tại sao Controlled Component quan trọng?

Controlled Component giúp React luôn biết dữ liệu hiện tại của form.

Ví dụ:

```
const [email, setEmail] = useState("");
```

React có thể kiểm tra:

```
if (!email.includes("@")) {  
  // email không hợp lệ  
}
```

Có thể:

- Validate dữ liệu.
- Hiển thị lỗi ngay lập tức.
- Disable/enable button.
- Format dữ liệu.
- Đồng bộ nhiều input.
- Hiển thị dữ liệu ở nơi khác.

Ví dụ:

```
<p>Email hiện tại: {email}</p>
```

PHẦN 4. FORM CƠ BẢN

Bước 9. Tạo form đăng ký

Tạo form gồm:

- Họ tên.
- Email.
- Mật khẩu.
- Giới tính.
- Sở thích.

Ví dụ:

```
import { useState } from "react";  
  
function App() {  
  const [name, setName] = useState("");  
  const [email, setEmail] = useState("");  
  
  const handleSubmit = (event) => {  
    event.preventDefault();  
  
    console.log({
```

```
    name,
    email
  });
};

return (
  <div>
    <h1>Đăng ký</h1>

    <form onSubmit={handleSubmit}>
      <div>
        <label>Họ tên:</label>

        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
      </div>

      <div>
        <label>Email:</label>

        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
      </div>

      <button type="submit">
        Đăng ký
      </button>
    </form>
  </div>
);
}

export default App;
```

Bước 10. Tại sao phải dùng event.preventDefault()?

Khi submit form HTML truyền thống, trình duyệt thường reload trang.

Trong React:

```
const handleSubmit = (event) => {  
  event.preventDefault();  
};
```

preventDefault() ngăn hành vi mặc định của trình duyệt.

Sau đó React có thể tự xử lý dữ liệu:

```
const handleSubmit = (event) => {  
  event.preventDefault();  
  
  console.log(name);  
  console.log(email);  
};
```

Đây là mô hình rất phổ biến trong ứng dụng React.

PHẦN 5. CONTROLLED FORM

Bước 11. Quản lý nhiều input

Không nên tạo quá nhiều state riêng biệt nếu form có nhiều trường.

Có thể dùng một object:

```
const [formData, setFormData] = useState({  
  name: "",  
  email: "",  
  password: ""  
});
```

Sau đó:

```
<input  
  name="name"  
  value={formData.name}  
  onChange={handleChange}  
/>
```

Bước 12. Tạo handleChange dùng chung

```
const handleChange = (event) => {  
  const { name, value } = event.target;  
  
  setFormData({  
    ...formData,  
    [name]: value  
  });  
};
```

Giải thích:

event.target: là input hiện tại.

name: xác định trường dữ liệu.

value: là giá trị người dùng nhập.

Ví dụ:

```
<input name="email" />
```

thì:

```
name === "email"
```

và React cập nhật:

```
formData.email
```

Bước 13. Hoàn thiện Controlled Form

```
import { useState } from "react";

function App() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: ""
  });

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value
    });
  };

  const handleSubmit = (event) => {
    event.preventDefault();

    console.log("Dữ liệu:", formData);
  };

  return (
```

```
<div>
  <h1>Đăng ký tài khoản</h1>

  <form onSubmit={handleSubmit}>
    <div>
      <label>Họ tên:</label>

      <input
        type="text"
        name="name"
        value={formData.name}
        onChange={handleChange}
      />
    </div>

    <div>
      <label>Email:</label>

      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
      />
    </div>

    <div>
      <label>Mật khẩu:</label>

      <input
        type="password"
        name="password"
        value={formData.password}
        onChange={handleChange}
      />
    </div>

    <button type="submit">
      Đăng ký
    </button>
  </form>

  <hr />
```

```
    <h3>Dữ liệu hiện tại</h3>

    <p>Họ tên: {formData.name}</p>
    <p>Email: {formData.email}</p>
  </div>
);
}

export default App;
```

PHẦN 6. CÁC LOẠI FORM ELEMENT

Bước 14. Checkbox

Checkbox thường sử dụng checked thay vì value

Ví dụ:

```
const [agree, setAgree] = useState(false);
<input
  type="checkbox"
  checked={agree}
  onChange={(e) => setAgree(e.target.checked)}
/>
```

Lấy trạng thái checkbox bằng e.target.checked

Bước 15. Radio Button

Ví dụ:

```
const [gender, setGender] = useState("male");
<label>
  <input
    type="radio"
    name="gender"
    value="male"
    checked={gender === "male"}
    onChange={(e) => setGender(e.target.value)}
  />
  Nam
</label>

<label>
  <input
    type="radio"
    name="gender"
```

```
value="female"
checked={gender === "female"}
onChange={(e) => setGender(e.target.value)}
/>
Nữ
</label>
```

Bước 16. Select

```
const [city, setCity] = useState("");
<select
  value={city}
  onChange={(e) => setCity(e.target.value)}
>
  <option value="">-- Chọn thành phố --</option>
  <option value="hanoi">Hà Nội</option>
  <option value="hcm">TP. Hồ Chí Minh</option>
  <option value="danang">Đà Nẵng</option>
</select>
```

PHẦN 7. UNCONTROLLED COMPONENT

Bước 17. Uncontrolled Component là gì?

Uncontrolled Component là form element mà giá trị hiện tại chủ yếu được DOM quản lý thay vì React state.

Thay vì: `value={name}` chúng ta có thể sử dụng: `defaultValue="Nguyễn Văn A"` và sử dụng `ref` để lấy giá trị khi cần.

Bước 18. Sử dụng `useRef`

Import:

```
import { useRef } from "react";
```

Tạo ref:

```
const nameRef = useRef();
```

Gắn vào input:

```
<input ref={nameRef} />
```

Khi submit:

```
const handleSubmit = (event) => {
  event.preventDefault();
```

```
    console.log(nameRef.current.value);  
  }  
};
```

Bước 19. Ví dụ Uncontrolled Form

```
import { useRef } from "react";  
  
function App() {  
  const nameRef = useRef();  
  const emailRef = useRef();  
  
  const handleSubmit = (event) => {  
    event.preventDefault();  
  
    const name = nameRef.current.value;  
    const email = emailRef.current.value;  
  
    console.log({  
      name,  
      email  
    });  
  };  
  
  return (  
    <div>  
      <h1>Uncontrolled Form</h1>  
  
      <form onSubmit={handleSubmit}>  
        <div>  
          <label>Họ tên:</label>  
  
          <input  
            type="text"  
            ref={nameRef}  
          />  
        </div>  
  
        <div>  
          <label>Email:</label>  
  
          <input  
            type="email"  
            ref={emailRef}  
          />  
        </div>  
      </form>  
    </div>  
  );  
}
```

```

        <button type="submit">
          Submit
        </button>
      </form>
    </div>
  );
}

export default App;

```

Ở đây React **không lưu từng giá trị input vào state**.

Giá trị được lấy trực tiếp từ DOM thông qua: `nameRef.current.value`

PHẦN 8. SO SÁNH CONTROLLED VS UNCONTROLLED

So sánh

Tiêu chí	Controlled	Uncontrolled
Nơi quản lý dữ liệu	React state	DOM
Thường dùng	<code>useState</code>	<code>useRef</code>
Đọc dữ liệu	State	<code>ref.current.value</code>
Theo dõi realtime	Dễ	Không thuận tiện
Validation realtime	Dễ	Khó hơn
Kiểm soát UI	Cao	Thấp hơn
Code	Có thể dài hơn	Đơn giản hơn
Form phức tạp	Rất phù hợp	Có thể dùng
Form đơn giản	Phù hợp	Phù hợp

Controlled

```

<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>

```

Uncontrolled

```
<input ref={nameRef} />
```

Lấy dữ liệu:

```
nameRef.current.value
```

Khi nào nên dùng Controlled?

Nên ưu tiên Controlled khi:

- Cần validation realtime.
- Cần hiển thị lỗi ngay khi nhập.
- Cần thay đổi UI dựa trên dữ liệu form.
- Cần disable button dựa trên dữ liệu.
- Form có nhiều logic xử lý.
- Cần đồng bộ dữ liệu giữa nhiều component.

Ví dụ:

```
<button disabled={!email}>  
  Submit  
</button>
```

Điều này rất dễ thực hiện khi sử dụng Controlled Component.

Khi nào có thể dùng Uncontrolled?

Uncontrolled phù hợp khi:

- Form tương đối đơn giản.
- Chỉ cần lấy dữ liệu khi submit.
- Không cần theo dõi từng thay đổi.
- Muốn giảm lượng state phải quản lý.

Ví dụ:

```
<form onSubmit={handleSubmit}>  
  <input ref={nameRef} />  
  <input ref={emailRef} />  
  
  <button type="submit">  
    Submit  
  </button>  
</form>
```

```
</button>  
</form>
```

PHẦN 9. BÀI TẬP THỰC HÀNH

Bài tập 1 – Counter

Tạo ứng dụng có:

Count: 0

[-] [Reset] [+]

Yêu cầu:

- Click + → tăng 1.
- Click - → giảm 1.
- Click Reset → về 0.
- Không cho count nhỏ hơn 0.

Sinh viên phải sử dụng:

- onClick
- useState

Bài tập 2 – Controlled Form

Xây dựng form đăng ký:

Họ tên
Email
Mật khẩu
Giới tính
Thành phố

[Đăng ký]

Yêu cầu:

1. Tất cả input phải là Controlled Component.
2. Dữ liệu được lưu trong useState.
3. Khi submit không reload trang.
4. Hiển thị dữ liệu đã nhập bên dưới form.

Ví dụ:

Thông tin đăng ký

Họ tên: Nguyễn Văn A
Email: nguyenvana@gmail.com
Giới tính: Nam
Thành phố: TP. Hồ Chí Minh

Bài tập 3 – Validation

Mở rộng bài tập 2.

Kiểm tra:

Họ tên

Không được để trống.

Email

Phải có dạng email hợp lệ.

Mật khẩu

Tối thiểu 6 ký tự.

Ví dụ:

```
if (formData.name.trim() === "") {  
  setError("Họ tên không được để trống");  
}
```

Bài tập 4 – Uncontrolled Form

Tạo một form:

Họ tên: _____
Email: _____
Số điện thoại: _____
[Submit]

Yêu cầu:

- Không sử dụng useState để lưu dữ liệu input.
- Sử dụng useRef.

- Khi submit, lấy toàn bộ dữ liệu.
 - Hiển thị dữ liệu ra console.
-

Bài tập 5 – So sánh hai phương pháp

Sinh viên xây dựng **hai form giống nhau**:

Form A

Sử dụng: Controlled Component

Form B

Sử dụng: Uncontrolled Component

Sau đó trả lời:

1. Dữ liệu được lưu ở đâu?
 2. Component có biết giá trị input trong quá trình nhập không?
 3. Phương pháp nào dễ validation realtime hơn?
 4. Phương pháp nào sử dụng useState?
 5. Phương pháp nào sử dụng useRef?
 6. Trong dự án thực tế, trường hợp nào nên dùng mỗi phương pháp?
-

PHẦN 10. BÀI TẬP TỔNG HỢP

Xây dựng Form đăng ký tài khoản

Thiết kế một ứng dụng:

=====

ĐĂNG KÝ TÀI KHOẢN

=====

Họ tên:

[_____]

Email:

[_____]

Mật khẩu:

[_____]

Giới tính:

☐ Nam ☐ Nữ

Thành phố:

[-- Chọn thành phố -- ▼]

☐ Tôi đồng ý với điều khoản

[ĐĂNG KÝ]

THÔNG TIN ĐÃ NHẬP

Họ tên: Nguyễn Văn A

Email: example@gmail.com

Giới tính: Nam

Thành phố: Hồ Chí Minh

=====

Yêu cầu chức năng

1. Họ tên: Không được rỗng.

2. Email

- Không được rỗng.
- Kiểm tra định dạng email.

3. Mật khẩu: Tối thiểu 6 ký tự.

4. Giới tính: Chọn Nam hoặc Nữ.

5. Thành phố: Có ít nhất 3 lựa chọn.

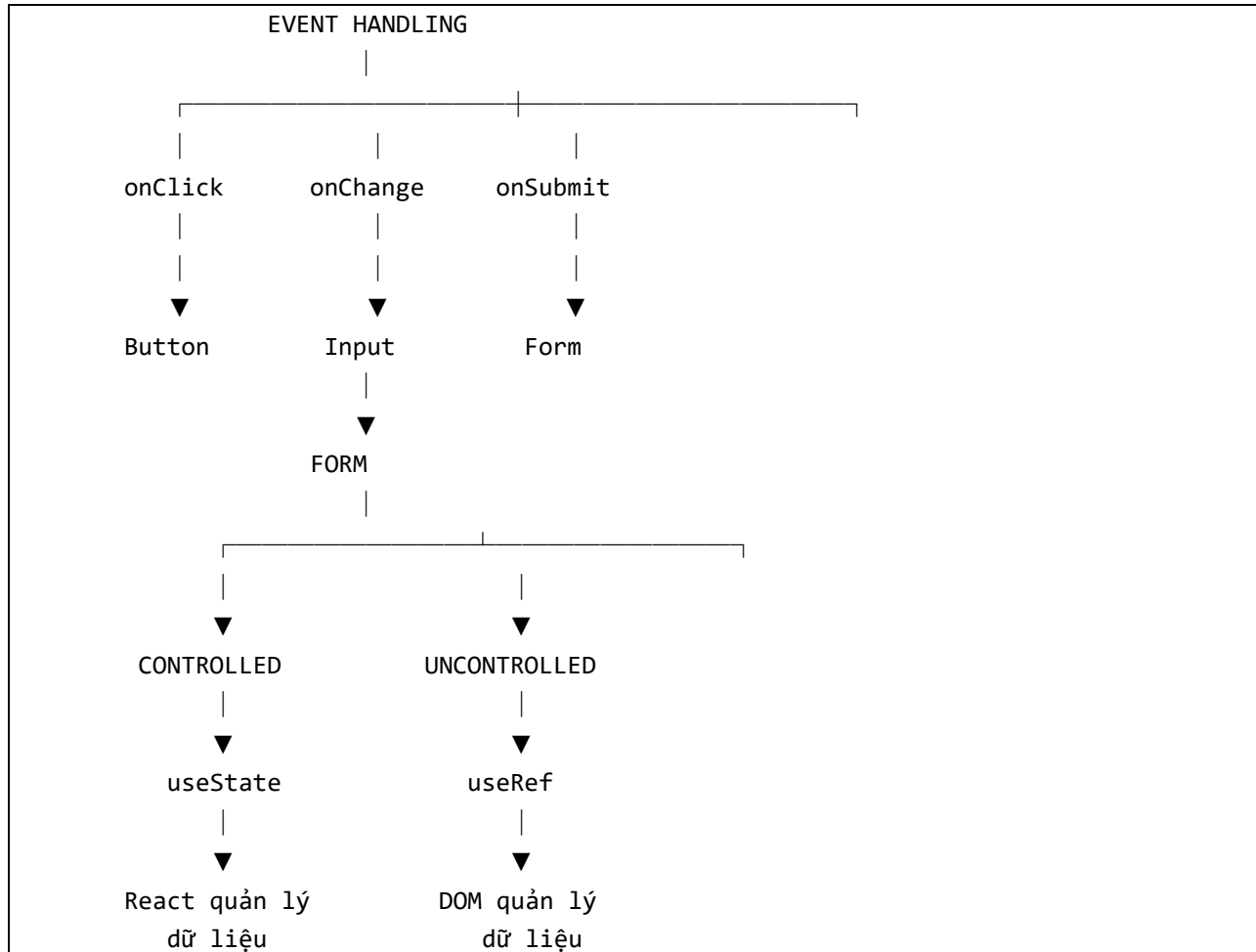
6. Điều khoản: Người dùng phải tick checkbox.

7. Submit

- Không reload trang.
- Kiểm tra dữ liệu.
- Nếu hợp lệ → hiển thị thông tin.
- Nếu không hợp lệ → hiển thị lỗi.

PHẦN 11. KIẾN THỨC CẦN GHI NHỚ

Có thể tóm tắt toàn bộ bài bằng sơ đồ:



Công thức quan trọng

Event: `<button onClick={handleClick}>`

Input: `<input onChange={handleChange} />`

Controlled:

```
const [value, setValue] = useState("");

<input
  value={value}
  onChange={(e) => setValue(e.target.value)}
/>
```

Uncontrolled

```
const inputRef = useRef();  
  
<input ref={inputRef} />  
Lấy dữ liệu:  
inputRef.current.value
```

Form Submit

```
const handleSubmit = (e) => {  
  e.preventDefault();  
};
```

PHẦN 12. CÂU HỎI KIỂM TRA

1. Event Handling trong React là gì?
2. onClick được sử dụng trong trường hợp nào?
3. onChange thường được sử dụng cho loại element nào?
4. event.target có ý nghĩa gì?
5. Tại sao không nên viết onClick={handleClick()} trong trường hợp thông thường?
6. event.preventDefault() có tác dụng gì?
7. Controlled Component là gì?
8. Controlled Component thường sử dụng hook nào?
9. Uncontrolled Component là gì?
10. useRef được sử dụng như thế nào với form?
11. Điểm khác nhau quan trọng nhất giữa Controlled và Uncontrolled Component là gì?
12. Trường hợp nào nên sử dụng Controlled Component?
13. Trường hợp nào có thể sử dụng Uncontrolled Component?
14. Checkbox lấy giá trị thông qua event.target.value hay event.target.checked?
15. Tại sao form phức tạp thường phù hợp với Controlled Component?